

**SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE**

**Fakulta elektrotechniky a informatiky**

Evidenčné číslo: FEI-xxxx-xxxx

**Rozšířená šablóna záverečnej práce na FEI STU  
v Bratislave v systéme  $\text{\LaTeX}$**

**Bakalárska práca**

**SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE**

**Fakulta elektrotechniky a informatiky**

Evidenčné číslo: FEI-xxxx-xxxx

**Rozšířená šablóna záverečnej práce na FEI STU  
v Bratislave v systéme  $\text{\LaTeX}$**

**Bakalárska práca**

|                      |                                                                |
|----------------------|----------------------------------------------------------------|
| Študijný program:    | Aplikovaná informatika                                         |
| Študijný odbor:      | Informatika                                                    |
| Školiace pracovisko: | Ústav multimediálnych informačných a komunikačných technológií |
| Školiteľ:            | Ing. Marek Vančo, PhD.                                         |

**Podakovanie**

# Abstrakt

## Kľúčové slová

záverečná práca, šablóna, L<sup>A</sup>T<sub>E</sub>X, formátovanie textu, citácie

# Abstract

## Keywords

Final thesis, template, L<sup>A</sup>T<sub>E</sub>X, text formatting, citations

# Obsah

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>Úvod</b>                                                             | <b>7</b>  |
| <b>1 Súčasný stav</b>                                                   | <b>8</b>  |
| 1.1 Znalostné grafy . . . . .                                           | 8         |
| 1.2 Klasické metódy extrakcie/extrakcia informácií z textu . . . . .    | 8         |
| 1.3 Spracovanie dokumentov a layout analýza . . . . .                   | 8         |
| 1.4 Existujúce prístupy pre tvorbu znalostných grafov z textu . . . . . | 8         |
| 1.5 Medze . . . . .                                                     | 8         |
| <b>2 Návrh riešenia</b>                                                 | <b>9</b>  |
| 2.1 Predspracovanie . . . . .                                           | 9         |
| 2.1.1 Detekcia rozloženia . . . . .                                     | 9         |
| 2.1.2 Extrakcia tabuliek . . . . .                                      | 10        |
| 2.2 Trojkroková extrakcia . . . . .                                     | 10        |
| 2.2.1 Detekcia v otvorenej doméne (ODD) . . . . .                       | 10        |
| 2.2.2 Úprava a spresnenie schémy (SR) . . . . .                         | 11        |
| 2.2.3 Schémou riadená extrakcia (SDE) . . . . .                         | 11        |
| 2.3 Vkladanie do databázy . . . . .                                     | 12        |
| <b>3 Implementácia</b>                                                  | <b>13</b> |
| 3.1 Technologický stack . . . . .                                       | 13        |
| 3.2 Kľúčové implementačné rozhodnutia . . . . .                         | 13        |
| 3.3 Prompt engineering pre slovenčinu . . . . .                         | 16        |
| <b>4 Experimenty a vyhodnotenie</b>                                     | <b>17</b> |
| 4.1 Testovanie dáta a experimentálne prostredie . . . . .               | 17        |
| 4.2 Priebeh extrakcie a kvantitatívne výsledky . . . . .                | 17        |
| 4.3 Vizualizácia znalostného grafu . . . . .                            | 17        |
| 4.4 Porovnanie s baseline . . . . .                                     | 17        |
| 4.5 Diskusia a limitácie . . . . .                                      | 17        |
| <b>Záver</b>                                                            | <b>18</b> |
| <b>Literatúra</b>                                                       | <b>19</b> |
| Použitie nástrojov umelej inteligencie . . . . .                        | 19        |

## Zoznam značiek a skratiek

# Úvod

# 1 Súčasný stav

8 s. dokopy

## 1.1 Znalostné grafy

1.5 s. Definícia, trojica (head, rel, tail), property graph vs. RDF, prečo Neo4j pre tento prípad, porovnanie s vector store a relačnými DB.

## 1.2 Klasické metódy extrakcie/extrakcia informácií z textu

1.5 s. Klasické NER/RE, prechod na LLM-based extraction, structured output, problém halucinácií a schéma-drift.

## 1.3 Spracovanie dokumentov a layout analýza

OCR a parsovanie PDF, problém tabuliek v plain-text extrakcii, DocLayout-YOLO ako riešenie detekcie štruktúrnych prvkov, vision LLM pre prevod obrázkov→štruktúrovaný výstup.

## 1.4 Existujúce prístupy pre tvorbu znalostných grafov z textu

3 s. najdôležitejšia sekcia

LangChain LLMGraphTransformer ako baseline ODKE+ pipeline (Open Domain Knowledge Extraction) — trojfázový návrh ODD → Schema Refinement → SDE GraphRAG (Microsoft) Špecifiká pre slovanské jazyky a právnu doménu (málo práce v tejto oblasti — tvoja medzera vo výskume)

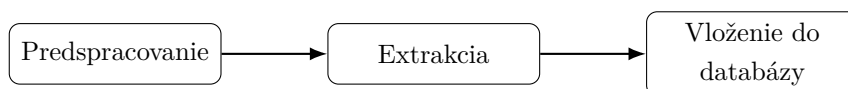
## 1.5 Medze

Čo existujúce prístupy neriešia: slovenčina, právna doména, hybridné spracovanie text + tabuľky v jednej pipeline → motivácia pre vlastný návrh.

## 2 Návrh riešenia

4 s. Blokový diagram funkcie `process()` — toto je tvoj najdôležitejší obrázok. PDF vstup → Load + Layout detection → dve paralelné vetvy (text a tabuľky) → ODD → Schema Refinement → SDE → Neo4j + Vector stores.

Spracovanie transformuje PDF dokument Slovenského právneho odvetvia z neštruktúrovaného textu na štruktúrované dáta, ktoré sa uložia v znalostnom grafe (KG). Je to súvislá troj-kroková postupnosť: predspracovanie, extrakcia a vloženie do databázy. Tento tok dát je znázornený na obrázku č.1.



Obr. 1: Graf popisujúci spracovanie.

### 2.1 Predspracovanie

Čisté spracovanie PDF dokumentu nie je dostačujúce pre legislatívne dokumenty. Napríklad tabuľky, ktoré sú nositeľmi dôležitých, opisných a rozvíjajúcich vlastností, ako napríklad v zákone 222/2004 Z. z. [1] na strane 159, kde je kapitola/číselný znak spoločného colného sadzobníka a k nemu podliehajúci opis tovaru. Dokonca, sa táto tabuľka rozkladá na viacero strán. Pri čítaní tabuľky, ako reťazca by sa tieto informácie roztrúsili a zničil by sa tak kontext celej tabuľky. Vzorce sa taktiež vyskytujú naprieč dokumentom (str. 149) a v PDF dokumente sú uchované ako obrázky - nástroj na čítanie textu z dokumentu by tento obrázok nerozpoznal a nenašiel.

#### 2.1.1 Detekcia rozloženia

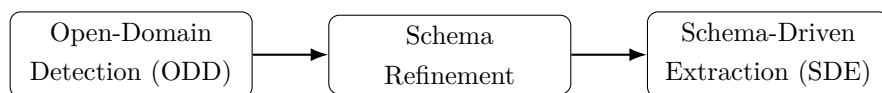
V tomto kroku sa každá strana dokumentu konvertuje na obrázok s rozlíšením 200 DPI. Aplikuje sa DocLayout-YOLO model, ktorý prejde a zdetekuje rozloženie každej strany. Tento detektor je nakonfigurovaný aby rozoznal päť cieľových tried: tabuľka, obrázok, diagram, izolovaný vzorec a popis vzorca. Detekcie s istotou pod 0,3 sú zahodené. Pre každú nájdenú detekciu, naväzujúci box je vystrihnutý z obrázku stránky a uložený ako bezstratový PNG súbor. Všetky susedné obrázky na základe strany (ak rozdiel strán je menší ako 2) sú zoskupené, aby tabuľky, ktoré sa rozliehajú na viacero strán, boli pri sebe ako jeden logický celok.

### 2.1.2 Extrakcia tabuliek

Každá skupina tabuliek je spracovaná v dvoch krokoch. Prvý krok, optický schopný Gemini model, príjme všetky orezané obrázky skupiny spoločne s system promptom, ktorý nariaďuje aby bol výstup tabuľka v podobe HTML. Tento prompt zakazuje sumarizáciu tabuľky a nariaďuje aby model zachoval doslovný text bunky. Druhý krok, výsledný HTML výstup je poskytnutý ďalšiemu LLM volaniu, kde tento krát ma model za úlohu transformovať tabuľu do hierarchickej podoby vrcholov a vzťahov: tabuľka je označená ako koreňový vrchol, bunky každého riadku prvého stĺpca ako rodič, a potomok pre každú ďalšiu bunku daného riadku. Je to cesta ako zachovať význam tabuľky v znalostnom grafe. Vnútorne strany viac-stranovej tabuľky sú vyradené zo zoznamu strán PDF dokumentu, pre neskoršiu extrakciu aby sa predišlo duplicitám rovnakej informácie.

## 2.2 Trojkroková extrakcia

Po dokončení predspracovania, zvyšné strany dokumentu sú rozsekané na chunky textov a spracované cez trojkrovú extrakciu inšpirovanú výskumom ODKE+ [2]. Táto extrakcia sa skladá z troch postupných krokov: detekcia v otvorenej doméne v a.j. „*open-domain detection*“ (ODD), úprava schémy v a.j. „*schema refinement*“ (SR) a v poslednom rade je to schémov riadená extrakcia, a.j. „*schema-driven extraction*“ (SDE). Tento tok je zobrazený na obrázku č. 2.



Obr. 2: Extrakcia inšpirovaná ODKE+ [2]

### 2.2.1 Detekcia v otvorenej doméne (ODD)

Cielom tohto prvého kroku je nájsť, aké typy entít a vzťahov sa v dokumente vyskytujú. Text je rozdelený pomocou nástroja RecursiveCharacterTextSplitter, s veľkosťou chunku 1200 charakterov a prekryvom 200 charakterov. Relatívne väčšie okno textu je zvolené schválne. V tejto etape model GPT-5.4-mini je vynútený aby vyhľadal typy a nie konkrétne inštancie. Širšie kontextové okno napomáha aby model videl viac ako len jednu vetu včase, čo je veľmi dôležité v legislačných dokumentoch, kde napríklad slovo „*základ dane*“ je uvedené niekoľko viet predtým ako je použité.

Každý chunk  $c_i$  je paralelne poslaný LLM agentovi, ktorého system prompt inštruuje aby našiel a vrátil všeobecné ale rozlíšiteľné typy entít a typy vzťahov. Agentovi je explicitne napísané aby sa zameriaval na typy ako sú zákony, paragrafy, práva a

povinnosti. Všetky diakritické znaky sú normalizované do ASCII aby znalostný graf používal jednotnú slovenskú výslovnosť bez diakritiky. Výstup tohto kroku je zjednotený zoznam schém chunkov:  $S_{ODD} = \bigcup_{i=1}^{|C|} S_i$ . Schému môžeme zobrazit ako  $S_i = (T_V^{(i)}, T_E^{(i)})$ , kde  $T_V^{(i)}$  sú typy entít (vrcholov v znalostnom grafe) a  $T_E^{(i)}$  sú typy vzťahov (hrán v znalostnom grafe) medzi entitami.

### 2.2.2 Úprava a spresnenie schémy (SR)

Zjednotené schémy z predošlého kroku detekcie (ODD), sú poslané na úpravu schémy modelu Gemini 2.5 Pro, spolu s existujúcou schémou znalostného grafu:  $S^* = \Phi(S_{ODD}, S_O)$ . Tento krok úpravy má čistou linguistickú úlohu: rieši duplikáty spôsobené dikaritikou alebo vynechaním písmena (Dan a Daň alebo ClenskyStat a ClenskStat) a v neposlednom rade zjednocuje sémanticky podobné typy a synonymá (Doklad, DanovyDoklad a HodnoveryDoklad zjednotí do Doklad). Zároveň nedovoľuje aby sa spojili sémanticky odlišné typy ako Dokument a Osoba. System prompt vynucuje pre typy vzťahov aby boli písané pomocou UPPER\_SNAKE\_CASE (SCREAMING\_SNAKE\_CASE) a pre typy entít PascalCase. Taktiež sa uchováva merge log, ktorý mapuje všetky typy do akého typu entity alebo vzťahu boli zjednotené. Pomáha to pri debuggovaní alebo auditovaní.

### 2.2.3 Schémou riadená extrakcia (SDE)

Posledný krok extrakcie je extrakcia pomocou natvrdo nanútenej schémy, v tomto prípade je to upravená  $S^*$  schéma. Text je v tomto kroku znovu rozdelený na menšie chunky s veľkosťou 1024 charakterov a prekryvom 128 charakterov. Tento menší chunk je podnietený odlišnou úlohou ako pri ODD. V tomto kroku GPT-5.4-mini model musí extrahovať konkrétne inštancie entít a vzťahov medzi nimi. Menšia veľkosť textu vynucuje model k extrahovaniu presnejších a lepšie uzemnených trojíc  $(e_h, r, e_t)$ , kde  $e_h$  je začiatková entita vzťahu,  $e_t$  je koncová entita vzťahu a  $r$  je vzťah medzi danými entitami. Menšie kontextové okno znižuje pravdepodobnosť, že sa dva nesúvisiace fakty zmiešajú do jedného vzťahu. Chunky sú posielané paralelne s limitovanou súbežnosťou a pause-and-retry mechanizmom, v prípade vyčerpania užívateľových tokenov za minútu.

Pre každý chunk  $c_i$ , model vracia uzly a vzťahy, ktoré korešpondujú upravenej schéme  $S^*$ . Týmto krokom získame GraphDocument  $G_i = (V_i, E_i, src_i)$ . Následne ľahká kontrola a opracovanie, znormalizuje id uzlov do title case a pre vzťahy do UPPER\_SNAKE\_CASE (SCREAMING\_SNAKE\_CASE), a vynechá trojice ktorým chýba zdroj alebo cieľ. Extrahované uzly z chunku sú spojené s uzlom *Chunk* pomocou vzťahu IN\_CHUNK. Neskôr vďaka tomuto vzťahu v kroku na získavanie dát, môžeme získať originálny text, ktorý podporuje dané trojice.

## 2.3 Wkładanie do databázy

## 3 Implementácia

3 s.

### 3.1 Technologický stack

Python, LangChain + langchain-neo4j, Neo4j s APOC, modely (Gemini, OpenAI pre SDE, vision modely), DocLayout-YOLO, pdf2image.

### 3.2 Kľúčové implementačné rozhodnutia

#### Mechanizmus semaforu

Pri spracovávaní dokumentov, ktoré sú rozdelené na desiatky či i stovky chunkov pomocou LLM modelu, nastávajú 2 problémy. Prvým problémom je, keď posielame veľa požiadaviek paralelne, tak prťažíme API a vyčerpáme limit tokenov za minútu (TPM - Tokens Per Minute). Druhým je, keď jedna požiadavka zlyhá kvôli týmto limitám, zvyšné požiadavky s istotou zlyhajú taktiež. Tieto problémy riešim pomocou koordinačného mechanizmu.

Inicializácia začína vytvorením 3 kontrolných prvkov. Prvým je semafor, ktorý riadi konkurenčný limit, koľko úloh naraz môže byť obsluhovaných. V tomto implementovaní je to 5 konkurenčných úloh, zatiaľ čo ostatné čakajú. Zabraňuje to systému aby neodoslal stovky požiadaviek naraz. Druhým prvkom je globálny prepínač *pause switch*, ktorý začína v pozícii *on*. Pokiaľ tento prepínač je v zapnutej pozícii, systém môže voľne posilať API požiadavky. Akonáhle prepínač sa dostane do pozície vypnutej, každá pracovná úloha v systéme sa zastaví na rovnakom synchronizačnom bode a čaká. Posledným prvkom je zámok, ktorý povoľuje v prípade chyby alebo zlyhania pracovať iba na jednej úlohe.

Po nastavení týchto riadiacich prvkov, je každému chunku pridelená asynchrónna úloha. Každá úloha sa riadi rovnakou rutinou a má povolené až tri pokusy na dokončenie svojho procesu. Na začiatku každého pokusu úloha naprv overí stav globálneho prepínača. Ak je prepínač vypnutý, úloha čaká, kým sa znova nezapne.

Keď je prepínač aktívny, úloha sa pokúsi získať miesto v semafore. Akonáhle miesto získa, spustí samotnú extrakcie alebo detekciu volaním LLM modelu. Ak požiadavka prebehne úspešne, úloha skončí a vráti svoj výsledok. Ak volanie zlyhá vyhodnotením výnimky a úloha má ešte k dispozícii ďalšie pokusy, nastáva situácia, kedy sa úloha pokúsi získať zámok. Ten zaručuje, že iba úplne prvá úloha, ktorá zlyhanie zaregistruje, sa stáva zodpovednou za jeho riešenie. Táto úloha následne vypne globálny prepínač, čím okamžite pozastaví všetky ostatné úlohy v systéme. Následne sa systém uspí na 60

sekúnd, čo je dostatočne dlhý čas na to, aby sa rate limit modelu obnovil, a následne sa prepínač opäť zapne.

Ak pracovná úloha nakoniec vyčerpá všetky tri pokusy bez úspechu, vzdá sa a propaguje výnimku ďalej, namiesto toho aby blokovala zvyšok systému. Medzitým boli všetky úlohy spustené v rovnakom okamihu, skrz mechanizmus task creation a počas celého procesu bežia paralelne v rámci jedného event loopu. Hlavná rutina čaká pomocou agregáčnej operácie gather, kým každá jednotlivá úloha neskončí.

Precíznosť tohto návrhu spočíva v spôsobe, akým zaobchádza so zlyhaním ako so zdieľaným signálom, nie ako s súkromnou udalosťou. Jediné zlyhanie je interpretované ako dôkaz toho, že celý spracovateľský systém je obmedzený, a preto je cooldown aplikovaný kolektívne a nie individuálne. Zámok, tu zohráva zásadnú rolu, pretože bez neho by mohli desiatky úloh detekovať to isté zlyhanie v rovnakom okamihu a spustiť svoj 60 sekundový interval nezávisle.

## Štruktúrovaný výstup

V kontexte spracovania prirodzeného jazyka prostredníctvom LLM predstavuje štruktúrovaný výstup (a. j. structured output) zásadnú techniku, ktorá zaručuje, že odpoveď volania modelu bude zodpovedať definovanej schéme. Vďaka tomu sa eliminuje chybovosť, ktorú zapríčiňuje parsovanie výstupných dát z modelu. Vďaka tomuto môžeme jednoducho deserializovať dáta do Python objektov (dictionary, dataclass alebo Pydantic modelu). V prostredí knižnice LangChain existujú dva odlišné spôsoby, ktoré majú svoje špecifické vlastnosti, výhody a obmedzenia v závislosti od použitého modelu.

Prvý prístup, ktorý sa v kóde uplatňuje predovšetkým na modely od spoločnosti OpenAI, je stratégia poskytovateľa (ProviderStrategy). Tento prístup využívajú modely, ktoré natívne podporujú štruktúrovaný výstup (OpenAI, Anthropic, xAI, Gemini)[3]. Schéma očakávaného výstupu sa pri tomto prístupe odovzdáva ako parameter formátu odpovede (response\_format) priamo pri vytváraní agenta. Samotná validácia prebieha na strane poskytovateľa, ktorý je o presnej štruktúre odpovede informovaný pred generovaním.

Model je následne obmedzený technikou nazývanou *constrained decoding*. Po vygenerovaní každého tokenu inferenčný engine určí, ktoré tokeny sú validné ako nasledujúce. Masku aplikuje na výber z rozdelenia pravdepodobnosti, čím efektívne znižuje pravdepodobnosť nevalidných tokenov na nulu [4].

Pri zapnutom parametri strict=True, OpenAI zaručuje 100%-nú zhodu s požadovanou schémou[4, 5].

Hlavnou výhodou tejto stratégie je deterministická záruka, že vrátený objekt bude validný voči zadanej schéme. Ďalším významným prínosom je integrácia s agentovým ekosystémom, čo umožňuje rozšírenie o nástroje, kontextovú pamäť a iteratívne uvažovanie bez nutnosti meniť spôsob, akým sa štruktúrovaný výstup spracováva. V kóde projektu sa tento prístup uplatňuje pri kľúčových úlohach, akými sú detekcia v otvorenej doméne a schémou riadená extrakcia.

Druhý prístup, použitý v projekte pri komunikácii s modelmi Gemini, je metóda priameho štruktúrovaného výstupu (`with_structured_output`). V kóde sa tento princíp kombinuje s metódou založenou na prevode schémy do formátu JSON Schema. Tento prístup pracuje na úrovni samotného modelu bez agentovej vrstvy. Metóda vracia nový `Runnable`, ktorý pri každom volaní vloží schému do natívneho parametra API (`response_format` pre OpenAI a pre Gemini `response_mime_type` alebo `response_json_schema`) a automaticky parsuje odpoveď [5, 6]. Ak je schéma Pydantic trieda, tak požiadavka vráti inštanciu tejto triedy, a pri JSON Schema zas vráti dict.

Pri modeloch Gemini je tento prístup obzvlášť efektívny, pretože tieto modely natívne podporujú generovanie odpovedí podľa špecifikácie JSON Schema na úrovni modelu [7]. Nie je teda potrebné riešiť problém na úrovni promptu ani sa spoliehať na následnú validáciu.

Výhodou tohto prístupu je jeho jednoduchosť volania a priamy návrat výsledku bez potreby extrahovania z vnoreného kontextového výstupu. V kóde sa tento prístup uplatňuje pri spracovávaní tabuliek do HTML, transformácií HTML tabuliek do grafovej reprezentácie a pri úprave schémy.

Obmedzením štruktúrovaného výstupu pri Gemini je, že model podporuje len podmnožinu špecifikácie JSON Schema a veľmi zložené alebo prehĺbené schémy môžu byť odmietnuté [7]. V takýchto prípadoch je potrebné schému zjednodušiť (kratšie názvy, menšia hĺbka vnorenia). Pri OpenAI zase platí, že režim `strict=True` vyžaduje `additionalProperties` nastavenú na `false` a všetky vlastnosti musia byť v zozname `required` [4].

## Normalizácia typov

camelCase properties, UPPER\_SNAKE\_CASE relácie, default Entity type, ASCII bez diakritiky

### 3.3 Prompt engineering pre slovenčinu

Dôraz na ASCII v názvoch typov (ako constraint pre property graph labels), chain-of-thought pokyny, few-shot príklady, riešenie skloňovania.

## 4 Experimenty a vyhodnotenie

9 s.

### 4.1 Testovanie dáta a experimentálne prostredie

1 s. Opis vybraných právnych textov (konkrétny zákon alebo colný sadzobník), štatistika vstupu (strany, tabuľky, odhadované entity), hardvér, použité verzie modelov.

### 4.2 Priebeh extrakcie a kvantitatívne výsledky

3 s.

Metriky z tvojho pipeline:

- Počet chunkov po oboch chunkovaniach - ODD: počet detegovaných typov uzlov a relácií per chunk, histogram - Refinement: kompresia schémy (napr. 150 surových typov → 45 kanonických), merge log s ukážkami ("Dan" + "Daň" + "dan" → "Dan") - SDE: počet extrahovaných inštancií uzlov a relácií - Tabuľková vetva: počet detegovaných tabuliek vs. manuálne spočítané, úspešnosť prevodu - Čas behu jednotlivých fáz, počet LLM volaní, odhad nákladov

### 4.3 Vizualizácia znalostného grafu

1.5 s. Ukážka subgrafu z Neo4j Browser (napr. PravnyPredpis → Paragraf → Polozka refazec), porovnanie vstupu (PDF stránka) s výstupom (zodpovedajúci subgraf).

### 4.4 Porovnanie s baseline

1 s. Napr. LLMGraphTransformer bez schema refinement kroku — ukáže pridanú hodnotu tvojho trojfázového návrhu. Alebo porovnanie s ODD-only (bez refinementu).

### 4.5 Diskusia a limitácie

1 s. Čo sa ukázalo ako silné stránky (konsolidácia schémy, hybridné spracovanie tabuliek), kde sú slabé miesta (náklady na LLM, závislosť na kvalite PDF, viacstránkové tabuľky s rozdielnou štruktúrou).

## Záver

# Literatúra

1. MINISTRY OF JUSTICE OF THE SLOVAK REPUBLIC. *Slov-Lex: Legal and Information Portal* [online]. Ministerstvo spravodlivosti Slovenskej republiky [cit. 2026-04-20]. Dostupné z: <https://www.slov-lex.sk/>.
2. KHORSHIDI, S. et al. *ODKE+: Ontology-guided open-domain knowledge extraction with LLMs* [online]. arXiv, Apple Inc., 2025 [cit. 2026-04-20]. Dostupné z eprint: 2509.04696.
3. LANGCHAIN, INC. *Structured output, LangChain documentation* [online]. LangChain, Inc. [cit. 2026-04-22]. Dostupné z: <https://docs.langchain.com/oss/python/langchain/structured-output>.
4. OPENAI. *Introducing Structured Outputs in the API* [online]. [cit. 2026-04-22]. Dostupné z: <https://openai.com/index/introducing-structured-outputs-in-the-api/>.
5. *LangChain OpenAI reference* [online]. [cit. 2026-04-22]. Dostupné z: [https://reference.langchain.com/python/langchain-openai/chat\\_models/base/BaseChatOpenAI/with\\_structured\\_output](https://reference.langchain.com/python/langchain-openai/chat_models/base/BaseChatOpenAI/with_structured_output).
6. *ChatGoogleGenerativeAI reference* [online]. [cit. 2026-04-22]. Dostupné z: [https://reference.langchain.com/python/langchain-google-genai/chat\\_models/ChatGoogleGenerativeAI/response\\_schema](https://reference.langchain.com/python/langchain-google-genai/chat_models/ChatGoogleGenerativeAI/response_schema).
7. GOOGLE. *Structured outputs — Gemini API* [online]. [cit. 2026-04-22]. Dostupné z: <https://ai.google.dev/gemini-api/docs/structured-output>.

## Použitie nástrojov umelej inteligencie